SECURE SOFTWARE DEVELOPMENT COURSE

ASSIGNMENT REPORT · FUZZING & STATIC ANALYSIS

Breaking & Fixing a Student Records Manager

How fuzz testing and static analysis uncovered 17 problems in a Python application — and how every one of them was fixed.

8
crashes & wrong results
found by the fuzzer

9
security issues
reported by Bandit

10/10
Pylint score
after the fixes

SUPERVISED BY

Dr. Yasser Foad
Secure Software Development

PREPARED BY

Abdelrahman Ahmed Abdelhamied	2406244656
Mahmoud Osama Elsayed	2406241599
Omar Muhammad Muhammed	2406241065

October 2026

Contents

01 Overview 3

02 Part 1 — Building the Program 4

03 Part 2a — Fuzz Testing 9

04 Part 2b — Static Analysis 13

05 Part 3 — Fixing the Problems 16

06 Re-testing the Fixed Program 22

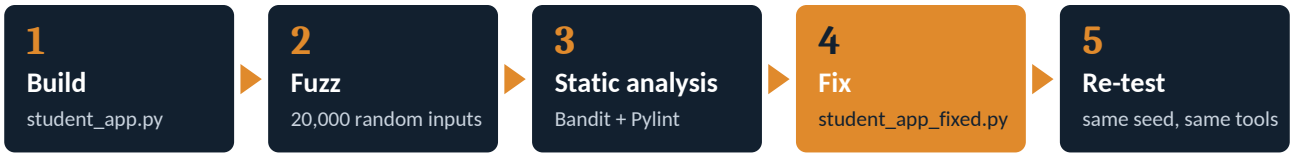
07 Fuzzing vs Static Analysis 24

08 Conclusion & Lessons Learned 25

01 Overview

Software that only works on *good* input is not secure software. The goal of this assignment is to see, in practice, how two different testing techniques find different kinds of problems in the same program, and what it takes to fix them.

The work was split into roles, like a small development team. One member acted as the **builder** and wrote the program. The other members acted as **testers**: they were not allowed to change the code, only to attack it and report what they found. Finally the team fixed the program together and re-ran every test.



Tools and environment

Tool	Type	What it was used for
Python 3	Language / runtime	The program under test and the fuzzer
Custom fuzzer (fuzz_task_1.py)	Dynamic testing	20,000 random and near-valid inputs against the core functions
Bandit 1.9	Static analysis (security)	Finds dangerous patterns: injection, eval, pickle, weak hashes…
Pylint	Static analysis (quality)	Coding-standard problems, risky constructs, code score out of 10
Kali Linux in VirtualBox	Environment	Where the static analysis was run and the reports saved

Assignment parts

Part	Role	Deliverable
Part 1	Builder	student_app.py - a working program with planted bugs and weaknesses
Part 2	Testers	Fuzzing results and Bandit / Pylint reports, with every finding explained
Part 3	Whole team	student_app_fixed.py, re-test results and a comparison of both techniques

02 Part 1 — Building the Program

The program under test is a command-line **Student Records Manager**. A user types a record in the format `name,age,grade1;grade2;grade3`, for example `Ali,20,90;85;77`. The program parses the record, calculates the average, converts it to a letter grade and saves the student in an SQLite database. A small menu also offers a search, a calculator and an admin-only database backup.

The code was written in two clearly separated areas, so that each testing technique has a realistic target:

- **Core logic** - `parse_record`, `average`, `letter_grade`, `make_username`, `process_line`. Pure functions that trust their input. These contain *robustness bugs* and are the fuzzer's target.
- **Storage and admin features** - database access, password handling, calculator and backup. These contain *security weaknesses* that are hard to hit with random input but easy to recognise in the source, the static analysers' target.

Planted problems

Where	Problem planted on purpose	Expected to be found by
parse_record	No check of field count, empty or non-numeric age / grades	Fuzzing
average	Division by zero for an empty grade list	Fuzzing
letter_grade	Score ≥ 100 overflows the table; negative score gives a wrong letter	Fuzzing
make_username	Crashes unless the name has exactly one space	Fuzzing
save_student / find_student	SQL built by string formatting (SQL injection)	Static analysis
hash_password	MD5 used for passwords	Static analysis
ADMIN_PASSWORD	Hard-coded password "admin123"	Static analysis
calculate	eval() on user input (code execution)	Static analysis
backup	shell=True with user input (command injection)	Static analysis
load_backup	pickle.load on a user-chosen file (code execution)	Static analysis
main	except Exception hides every error	Static analysis

student_app.py - static-analysis-lab - Visual Studio Code

py student_app.py

```
1  """
2  Student Records Manager  (Part 1 - the program under test)
3
4  A small command-line program that stores students and their grades.
5
6  Input format for a student record:   name,age,grade1;grade2;grade3
7  Example:                              Ali,20,90;85;77
8
9  NOTE FOR THE COURSE: this program contains bugs and security weaknesses
10 ON PURPOSE so that the testers (fuzzing + static analysis) can find them.
11 Do NOT use this code in a real system.
12 """
13
14 import hashlib
15 import pickle
16 import sqlite3
17 import subprocess
18
19 # Weakness: hard-coded admin password in the source code
20 ADMIN_PASSWORD = "admin123"
21
22 DB_FILE = "students.db"
23
24 # Letter grade table: index 0 = 0-9, 1 = 10-19, ... 9 = 90-99
25 LETTERS = ["F", "F", "F", "F", "F", "F", "D", "C", "B", "A"]
```

main ⊗ 0 ⚠ 0 Ln 1, Col 1 Spaces: 4 UTF-8 LF Python 3.13

FIGURE 1 student_app.py, lines 1–26: module header, risky imports (pickle, subprocess) and the hard-coded admin password.

student_app.py - static-analysis-lab - Visual Studio Code

py student_app.py

```
28 # ----------------------------------------------------------------------
29 # Core logic (these are the functions the fuzzer attacks)
30 # ----------------------------------------------------------------------
31
32 def parse_record(line):
33     """Turn 'name,age,g1;g2;g3' into a dictionary."""
34     parts = line.split(",")
35     name = parts[0]
36     age = int(parts[1])              # Bug: crashes if age is missing or not a number
37     grades = [int(g) for g in parts[2].split(";")]   # Bug: crashes on bad grades
38     return {"name": name, "age": age, "grades": grades}
39
40
41 def average(grades):
42     """Average of a list of grades."""
43     return sum(grades) / len(grades)  # Bug: division by zero on empty list
44
45
46 def letter_grade(score):
47     """Convert a numeric score to a letter."""
48     return LETTERS[score // 10]       # Bug: score 100+ -> IndexError,
49                                       #      negative score -> wrong letter
50
51
52 def make_username(name):
53     """Build a short username from the first 3 letters of the first and last name."""
54     first, last = name.split(" ")     # Bug: crashes unless exactly ONE space
55     return (first[:3] + last[:3]).lower()
56
57
58 def process_line(line):
59     """Full pipeline used by the app: parse -> average -> letter."""
60     record = parse_record(line)
61     avg = average(record["grades"])
62     return record["name"], avg, letter_grade(int(avg))
```

main ⊗ 0 ⚠ 0 Ln 1, Col 1 Spaces: 4 UTF-8 LF Python 3.13

FIGURE 2 Core logic, lines 28–63. Highlighted lines are the planted input-handling bugs that the fuzzer will attack.

student_app.py - static-analysis-lab - Visual Studio Code

py student_app.py

```
65 # ---------------------------------------------------------------------------
66 # Storage / admin features (these are where static analysis finds problems)
67 # ---------------------------------------------------------------------------
68
69 def save_student(record):
70     conn = sqlite3.connect(DB_FILE)
71     conn.execute("CREATE TABLE IF NOT EXISTS students (name TEXT, age INT)")
72     # Weakness: SQL built with string formatting -> SQL injection
73     conn.execute("INSERT INTO students VALUES ('%s', %d)" % (record["name"], record["age"]))
74     conn.commit()
75     conn.close()
76
77
78 def find_student(name):
79     conn = sqlite3.connect(DB_FILE)
80     # Weakness: SQL injection again
81     rows = conn.execute("SELECT * FROM students WHERE name = '" + name + "'").fetchall()
82     conn.close()
83     return rows
84
85
86 def hash_password(password):
87     # Weakness: MD5 is a broken hash for passwords
88     return hashlib.md5(password.encode()).hexdigest()
89
90
91 def check_admin(password):
92     return password == ADMIN_PASSWORD
93
94
95 def calculate(expression):
96     # Weakness: eval() runs ANY Python code the user types
97     return eval(expression)
```

main ⊗ 0 ⚠ 0 Ln 1, Col 1 Spaces: 4 UTF-8 LF Python 3.13

FIGURE 3 Storage and admin features, lines 65–98: SQL injection (73, 81), MD5 (88), plain-text password comparison (92) and eval (97).

student_app.py - static-analysis-lab - Visual Studio Code

py student_app.py

```
 99 def backup(filename):
100     # Weakness: user input passed to the shell -> command injection
101     subprocess.call("copy " + DB_FILE + " " + filename, shell=True)
102
103
104 def load_backup(path):
105     # Weakness: unpickling untrusted data can execute code
106     with open(path, "rb") as f:
107         return pickle.load(f)
108
109
110 # ---------------------------------------------------------------------------
111 # Simple menu
112 # ---------------------------------------------------------------------------
113
114 def main():
115     while True:
116         print("\n1) Add student  2) Find student  3) Calculator  4) Backup (admin)  5) Quit")
117         choice = input("> ").strip()
118         try:
119             if choice == "1":
120                 line = input("name,age,grades: ")
121                 save_student(parse_record(line))
122                 name, avg, letter = process_line(line)
123                 print(f"Saved {name}: average {avg:.1f} ({letter})")
124             elif choice == "2":
125                 print(find_student(input("name: ")))
126             elif choice == "3":
127                 print(calculate(input("expression: ")))
128             elif choice == "4":
129                 if check_admin(input("admin password: ")):
130                     backup(input("backup file name: "))
131                     print("Backup done")
132                 else:
133                     print("Wrong password")
134             elif choice == "5":
135                 break
136         except Exception as e:
137             print("Error:", e)
138
139
140 if __name__ == "__main__":
141     main()
142
```

main ⊗ 0 ⚠ 0 Ln 1, Col 1 Spaces: 4 UTF-8 LF Python 3.13

FIGURE 4 Lines 99–142: shell command injection (102), unsafe pickle loading (108) and the menu with a catch-all exception handler (137).

03 Part 2a — Fuzz Testing

Fuzzing is a **dynamic** technique: the program is executed over and over with automatically generated input, and every crash or wrong answer is recorded. It needs no knowledge of how the code is written, only a way to call it.

How the fuzzer works

- **20,000 iterations** with a fixed random seed (`random.seed(1)`), so the run is repeatable and the results can be checked by anyone.
- **Two kinds of input**, alternating: completely random text, and *near-valid* records built from a list of tricky values (empty fields, negative ages, `999999`, grades such as 150 or "x"). Near-valid input passes the first checks and reaches deeper code, which pure noise rarely does.
- **Every core function** is called inside `try / except`; the first input that raises each (function, exception) pair is stored, so the report contains unique findings, not 20,000 duplicates.
- **An oracle** checks a bug that does *not* crash: a negative score must always give "F". Without it, a wrong result would go unnoticed.

fuzz_task_1.py - static-analysis-lab - Visual Studio Code

py fuzz_task_1.py

```python
import random
import string
import student_app as app

random.seed(1)          # so the results repeat the same every run
found = {}              # (function, error type) -> first input that broke it


def record(func, exc, data):
    key = (func, type(exc).__name__)
    if key not in found:
        found[key] = (data, str(exc))


def rand_text():
    chars = string.ascii_letters + string.digits + ",;. -'\""
    return "".join(random.choice(chars) for _ in range(random.randint(0, 25)))


def rand_record():
    # text close to the valid format to reach deeper code
    name = random.choice(["Ali", "Sara Ahmed", "Omar Bin Ali", "", "A B"])
    age = random.choice(["20", "-5", "abc", "", "999999"])
    grades = ";".join(
        random.choice(["90", "85", "100", "-5", "x", "", "150"])
        for _ in range(random.randint(0, 4))
    )
    return f"{name},{age},{grades}"
```

main ⊗ 0 ⚠ 0 Ln 1, Col 1 Spaces: 4 UTF-8 LF Python 3.13

FIGURE 5 fuzz_task_1.py, lines 1–30: setup, finding recorder and input generators (comments translated to English).

fuzz_task_1.py - static-analysis-lab - Visual Studio Code

py fuzz_task_1.py

```
31 for i in range(20000):
32     s = rand_text() if i % 2 else rand_record()
33     n = random.randint(-300, 300)
34     grades = [random.randint(-200, 200) for _ in range(random.randint(0, 5))]
35
36     tests = [
37         ("parse_record",  s,      lambda: app.parse_record(s)),
38         ("process_line",  s,      lambda: app.process_line(s)),
39         ("make_username", s,      lambda: app.make_username(s)),
40         ("letter_grade",  n,      lambda: app.letter_grade(n)),
41         ("average",       grades, lambda: app.average(grades)),
42     ]
43     for name, inp, fn in tests:
44         try:
45             fn()
46         except Exception as e:
47             record(name, e, inp)
48
49     # oracle for the silent bug: negative score (-1..-10) must give F
50     if -10 <= n < 0:
51         try:
52             r = app.letter_grade(n)
53             if r != "F":
54                 record("letter_grade (wrong result)",
55                        AssertionError(f"negative {n} gave {r}"), n)
56         except Exception:
57             pass   # the IndexError was already recorded above
58
59
60 print("=== UNIQUE FINDINGS ===")
61 for (f, e), (inp, msg) in found.items():
62     print(f"{f:28} {e:20} input={inp!r}  msg={msg}")
```

main ⊗ 0 ⚠ 0 Ln 1, Col 1 Spaces: 4 UTF-8 LF Python 3.13

FIGURE 6 fuzz_task_1.py, lines 31–62: the main loop, the test table and the wrong-result oracle.

kali@kali: ~/static-analysis-lab

Session Actions Edit View Help

```
┌──(kali@kali)-[~/static-analysis-lab]
└─$ python3 fuzz_task_1.py
=== UNIQUE FINDINGS ===
parse_record                 ValueError          input='Sara Ahmed,999999,'  msg=invalid literal for int() with base 10: ''
process_line                 ValueError          input='Sara Ahmed,999999,'  msg=invalid literal for int() with base 10: ''
average                      ZeroDivisionError   input=[]  msg=division by zero
make_username                ValueError          input='58WAm,dX3a5IDnO'  msg=not enough values to unpack (expected 2, got 1)
letter_grade                 IndexError          input=-269  msg=list index out of range
parse_record                 IndexError          input='4;DSDC6'  msg=list index out of range
process_line                 IndexError          input='4;DSDC6'  msg=list index out of range
letter_grade (wrong result)  AssertionError      input=-4  msg=negative -4 gave A
┌──(kali@kali)-[~/static-analysis-lab]
└─$
```

FIGURE 7 Running the fuzzer: 8 unique findings in the original program.

Fuzzing results

#	Function	Exception	Failing input	Root cause
1	parse_record	ValueError	`'Sara Ahmed,999999,'`	Empty grade text: int('') fails
2	process_line	ValueError	`'Sara Ahmed,999999,'`	Same bug, reached through the pipeline
3	average	ZeroDivisionError	`[]`	len(grades) is 0
4	make_username	ValueError	`'58WAm,dX3a5IDnO'`	No space in the name, unpacking fails
5	letter_grade	IndexError	`-269`	-269 // 10 = -27, outside the table
6	parse_record	IndexError	`'4;DSDC6'`	Fewer than three comma-separated fields
7	process_line	IndexError	`'4;DSDC6'`	Same bug, reached through the pipeline
8	letter_grade	Wrong result	`-4`	-4 // 10 = -1, LETTERS[-1] is “A”

KEY OBSERVATION

Finding 8 is the most dangerous one. The program does not crash, it quietly gives a failing student an **A**. Only the oracle caught it, which shows that fuzzing finds logic errors only when the tester also tells it what the correct answer should be.

The fuzzer did **not** hit the score-100 bug (`100 // 10 = 10` is past the end of the list), because random averages of near-valid grades almost never land exactly on 100 or above. Random testing can miss narrow edge cases; this one was found by reading the code and is fixed anyway. The fuzzer also never touched SQL, eval, the shell or pickle, because it only called the core functions.

04 Part 2b — Static Analysis

Static analysis reads the source code **without running it** and compares it against known dangerous patterns and coding rules. It sees every line, including code that random input never reaches.

Bandit - security linter

Bandit parses the Python code into a syntax tree and checks it against a list of insecure patterns, each linked to a CWE weakness ID. It was run on Kali Linux and the report saved in text and JSON form:

COMMANDS USED

```
bandit -f txt -o bandit_report.txt student_manager.py
bandit -f json -o bandit_report.json student_manager.py
pylint student_manager.py > pylint_report.txt 2>&1
```

Line	Test	Severity	Confidence	Problem	CWE
102	B602	High	High	subprocess call with shell=True - command injection	78
88	B324	High	High	Weak MD5 hash used for passwords	327
73	B608	Medium	Medium	SQL query built with % formatting - SQL injection	89
81	B608	Medium	Medium	SQL query built with + concatenation - SQL injection	89
97	B307	Medium	High	eval() on user input - arbitrary code execution	78
108	B301	Medium	High	pickle.load on an untrusted file - code execution	502
20	B105	Low	Medium	Hard-coded password string 'admin123'	259
15	B403	Low	High	import pickle	502
17	B404	Low	High	import subprocess	78

In total Bandit reported **9 issues**: 2 High, 4 Medium and 3 Low severity (by confidence: 6 High, 3 Medium). The two Low “import” warnings are informational: importing pickle or subprocess is not a flaw by itself, but it tells the reviewer where to look.

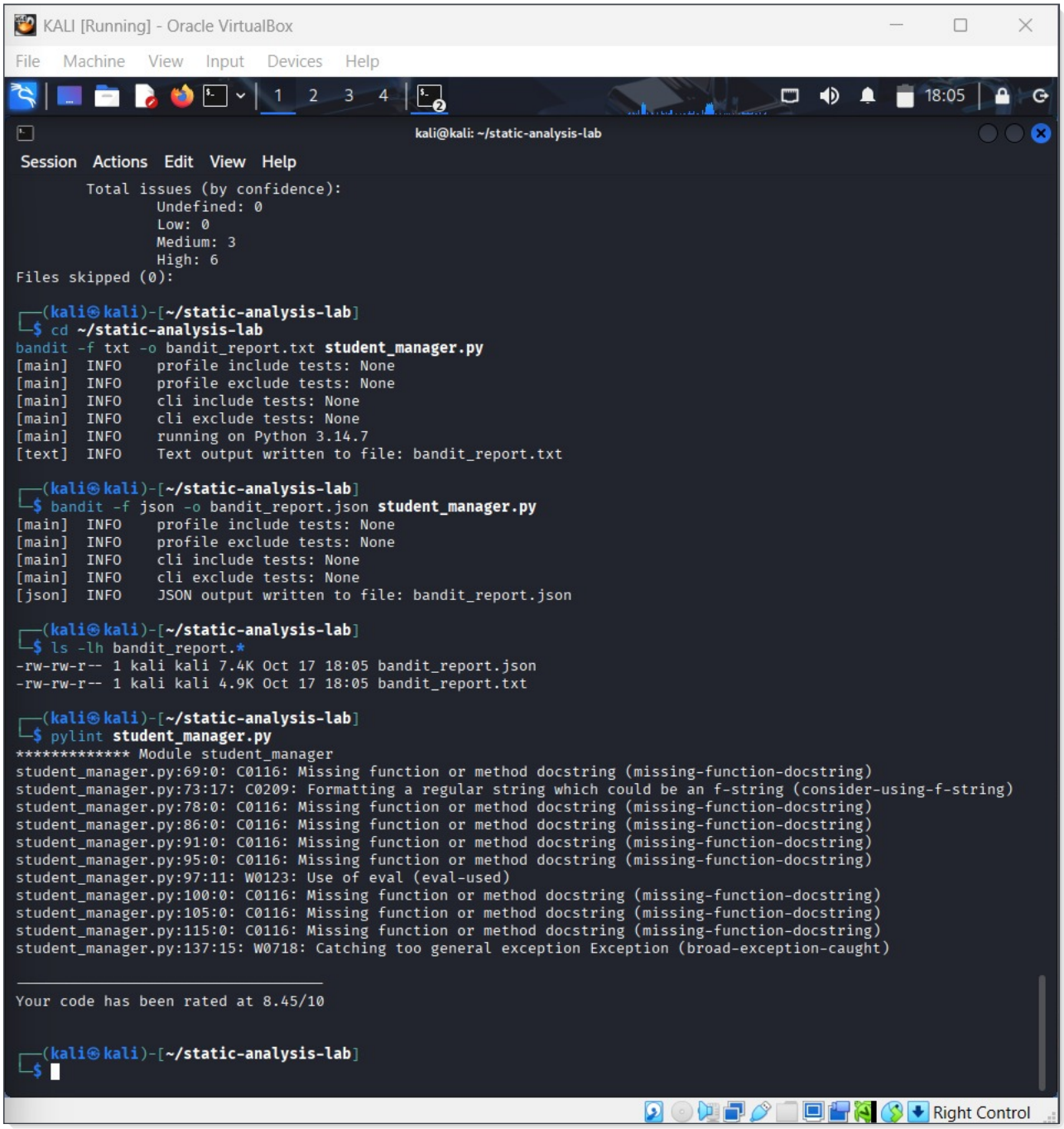


FIGURE 8 Bandit summary and report files on Kali Linux, followed by the first Pylint run on the same file.

Pylint - code quality

Pylint checks coding standards and risky constructs. It rated the program **8.45 / 10** with 12 messages. Two of them (eval-used and broad-exception-caught) point at the same weaknesses that matter for security; the rest are maintainability issues.

Code	Count	Message	Lines
C0116	9	missing-function-docstring	69, 78, 86, 91, 95, 100, 105, 115
C0209	1	consider-using-f-string	73 - the same line as the SQL injection
W0123	1	eval-used	97
W0718	1	broad-exception-caught (except Exception)	137

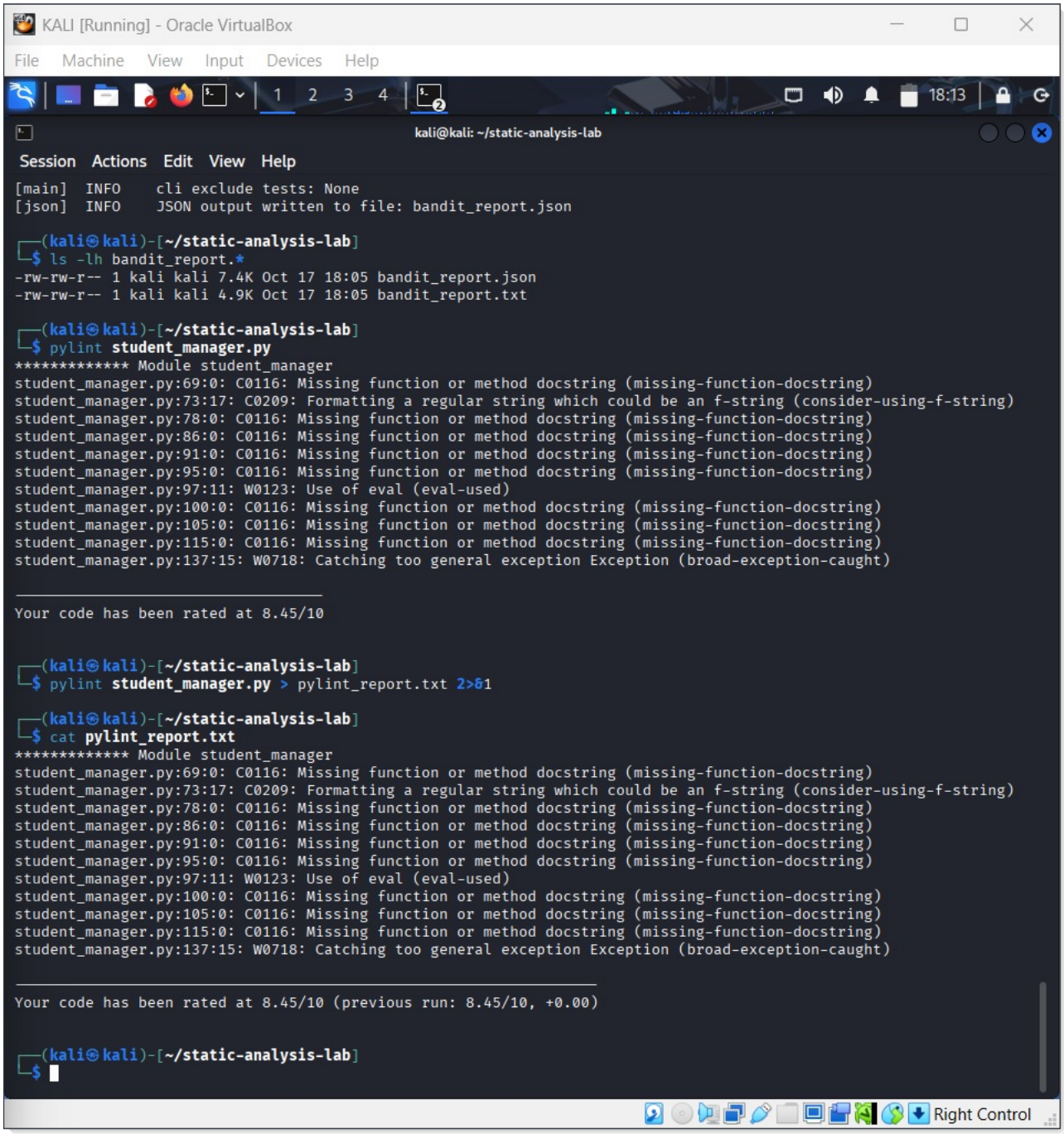


FIGURE 9 Pylint output saved to pylint_report.txt - score 8.45 / 10.

05 Part 3 — Fixing the Problems

All findings were fixed in a new file, **student_app_fixed.py**. The guiding rule was to fix the *cause*, not the symptom: validate input once at the boundary, and stop building code, SQL or shell commands out of strings.

Bad user input is now rejected with a dedicated exception, `InvalidRecordError` (a subclass of ValueError). The menu catches it and shows a clear message, so invalid input is an expected, controlled outcome instead of a crash.

Problem	Found by	Fix applied
parse_record crashes on missing / bad fields	Fuzzer	Exactly 3 fields, non-empty name, age 1–120, every grade 0–100
Division by zero in average	Fuzzer	Empty grade list is rejected
letter_grade IndexError and wrong “A”	Fuzzer	Scores outside 0–100 rejected; min(score // 10, 9) so 100 = A
make_username needs exactly one space	Fuzzer	name.split() on any whitespace; first and last word used
SQL injection (2 places)	Bandit B608	Parameterised queries with ? placeholders
Command injection via shell=True	Bandit B602, B404	shutil.copyfile - no shell; file name reduced to a safe base name
eval() calculator	Bandit B307, Pylint W0123	Small AST evaluator: only numbers and + − × ÷
pickle.load backup	Bandit B301, B403	Backups stored and loaded as JSON (data only)
MD5 password hash	Bandit B324	Salted PBKDF2-HMAC-SHA256, 200,000 iterations
Hard-coded admin password	Bandit B105	Hash read from the ADMIN_PASSWORD_HASH environment variable, hmac.compare_digest
Docstrings, f-string, broad except	Pylint	Docstrings added; only specific exceptions are caught
Student saved before validation	Code review	Record is validated before save_student is called

student_app_fixed.py - static-analysis-lab - Visual Studio Code

py student_app_fixed.py

```python
class InvalidRecordError(ValueError):
    """Raised on purpose when the user gives bad input (a controlled error)."""


# ---------------------------------------------------------------------------
# Core logic
# ---------------------------------------------------------------------------

def parse_record(line):
    """Turn 'name,age,g1;g2;g3' into a dictionary, validating every field."""
    parts = line.split(",")
    if len(parts) != 3:
        raise InvalidRecordError("expected exactly 3 fields: name,age,grades")
    name, age_text, grades_text = (p.strip() for p in parts)
    if not name:
        raise InvalidRecordError("name is empty")
    if not age_text.isdigit() or not 1 <= int(age_text) <= 120:
        raise InvalidRecordError("age must be a whole number between 1 and 120")
    grades = []
    for g in grades_text.split(";"):
        g = g.strip()
        if not g.isdigit() or int(g) > 100:
            raise InvalidRecordError(f"bad grade {g!r} (must be 0-100)")
        grades.append(int(g))
    return {"name": name, "age": int(age_text), "grades": grades}
```

main ⊗ 0 ⚠ 0 Ln 1, Col 1 Spaces: 4 UTF-8 LF Python 3.13

FIGURE 10 Fixed parse_record: every field is validated before use (green lines are new checks).

student_app_fixed.py - static-analysis-lab - Visual Studio Code

py student_app_fixed.py

```python
def average(grades):
    """Average of a non-empty list of grades."""
    if not grades:
        raise InvalidRecordError("cannot average an empty list of grades")
    return sum(grades) / len(grades)


def letter_grade(score):
    """Convert a numeric score (0-100) to a letter."""
    if not 0 <= score <= 100:
        raise InvalidRecordError(f"score {score} is outside 0-100")
    return LETTERS[min(score // 10, 9)]          # 100 maps to 'A'


def make_username(name):
    """Build a username from the first 3 letters of the first and last name."""
    words = name.split()                          # any amount of whitespace
    if len(words) < 2:
        raise InvalidRecordError("name needs a first and a last name")
    return (words[0][:3] + words[-1][:3]).lower()


def process_line(line):
    """Full pipeline used by the app: parse -> average -> letter."""
    record = parse_record(line)
    avg = average(record["grades"])
    return record["name"], avg, letter_grade(int(avg))
```

main ⊗ 0 ⚠ 0 Ln 1, Col 1 Spaces: 4 UTF-8 LF Python 3.13

FIGURE 11 Fixed average, letter_grade, make_username and process_line.

student_app_fixed.py - static-analysis-lab - Visual Studio Code

py student_app_fixed.py

```python
# ---------------------------------------------------------------------------
# Storage / admin features
# ---------------------------------------------------------------------------

def save_student(record):
    """Insert a student using a parameterised query (no SQL injection)."""
    with sqlite3.connect(DB_FILE) as conn:
        conn.execute("CREATE TABLE IF NOT EXISTS students (name TEXT, age INT)")
        conn.execute("INSERT INTO students VALUES (?, ?)",
                     (record["name"], record["age"]))


def find_student(name):
    """Look a student up by name using a parameterised query."""
    with sqlite3.connect(DB_FILE) as conn:
        return conn.execute("SELECT * FROM students WHERE name = ?",
                            (name,)).fetchall()


def hash_password(password, salt=None):
    """Salted PBKDF2-SHA256 instead of MD5. Returns 'salt$hash' in hex."""
    salt = salt or os.urandom(16)
    digest = hashlib.pbkdf2_hmac("sha256", password.encode(), salt, 200_000)
    return salt.hex() + "$" + digest.hex()


def check_admin(password):
    """Compare against the stored hash in constant time."""
    if "$" not in ADMIN_PASSWORD_HASH:
        return False
    salt_hex, _ = ADMIN_PASSWORD_HASH.split("$", 1)
    candidate = hash_password(password, bytes.fromhex(salt_hex))
    return hmac.compare_digest(candidate, ADMIN_PASSWORD_HASH)
```

main ⊗ 0 ⚠ 0 Ln 1, Col 1 Spaces: 4 UTF-8 LF Python 3.13

FIGURE 12 Parameterised SQL, salted PBKDF2 password hashing and a constant-time admin check.

student_app_fixed.py - static-analysis-lab - Visual Studio Code

py student_app_fixed.py

```python
_OPS = {ast.Add: operator.add, ast.Sub: operator.sub, ast.Mult: operator.mul,
        ast.Div: operator.truediv, ast.USub: operator.neg, ast.UAdd: operator.pos}


def calculate(expression):
    """Safe calculator: only numbers and + - * / are allowed (no eval)."""
    def walk(node):
        if isinstance(node, ast.Constant) and isinstance(node.value, (int, float)):
            return node.value
        if isinstance(node, ast.BinOp) and type(node.op) in _OPS:
            return _OPS[type(node.op)](walk(node.left), walk(node.right))
        if isinstance(node, ast.UnaryOp) and type(node.op) in _OPS:
            return _OPS[type(node.op)](walk(node.operand))
        raise InvalidRecordError("only numbers and + - * / are allowed")
    if len(expression) > 100:
        raise InvalidRecordError("expression too long")
    return walk(ast.parse(expression, mode="eval").body)


def backup(filename):
    """Copy the database with shutil - no shell, and the name is sanitised."""
    safe_name = Path(filename).name               # strips any ../ or /path
    if not safe_name or not safe_name.replace(".", "").replace("_", "").isalnum():
        raise InvalidRecordError("backup name may only contain letters, digits, '.' and '_'")
    BACKUP_DIR.mkdir(exist_ok=True)
    shutil.copyfile(DB_FILE, BACKUP_DIR / safe_name)


def load_backup(path):
    """Load backup data as JSON (data only - cannot execute code like pickle)."""
    with open(path, "r", encoding="utf-8") as f:
        return json.load(f)
```

main ⊗ 0 ⚠ 0 Ln 1, Col 1 Spaces: 4 UTF-8 LF Python 3.13

FIGURE 13 eval replaced by a safe AST calculator, shell replaced by shutil with a sanitised name, pickle replaced by JSON.

student_app_fixed.py - static-analysis-lab - Visual Studio Code

py student_app_fixed.py

```
155 # ------------------------------------------------------------------------
156 # Simple menu
157 # ------------------------------------------------------------------------
158
159 def main():
160     """Interactive menu loop."""
161     while True:
162         print("\n1) Add student  2) Find student  3) Calculator  4) Backup (admin)  5) Quit")
163         choice = input("> ").strip()
164         try:
165             if choice == "1":
166                 line = input("name,age,grades: ")
167                 name, avg, letter = process_line(line)   # validate BEFORE saving
168                 save_student(parse_record(line))
169                 print(f"Saved {name}: average {avg:.1f} ({letter})")
170             elif choice == "2":
171                 print(find_student(input("name: ")))
172             elif choice == "3":
173                 print(calculate(input("expression: ")))
174             elif choice == "4":
175                 if check_admin(input("admin password: ")):
176                     backup(input("backup file name: "))
177                     print("Backup done")
178                 else:
179                     print("Wrong password")
180             elif choice == "5":
181                 break
182         except (InvalidRecordError, SyntaxError, ZeroDivisionError) as e:
183             print("Invalid input:", e)
184         except (sqlite3.Error, OSError) as e:
185             print("Storage error:", e)
186
187
188 if __name__ == "__main__":
189     main()
```

main ⊗ 0 ⚠ 0 Ln 1, Col 1 Spaces: 4 UTF-8 LF Python 3.13

FIGURE 14 The menu validates before saving and catches only specific exceptions.

06 Re-testing the Fixed Program

A fix is only proven when the same test that found the problem passes. The fuzzer was run again with the **same seed and the same 20,000 iterations**; the only change is that the controlled `InvalidRecordError` counts as correct behaviour. Bandit and Pylint were run again with the same settings.

FUZZER FINDINGS	BANDIT ISSUES	PYLINT SCORE
8 → 0	9 → 0	8.45 → 10.00
original → fixed	original → fixed	original → fixed

Valid input still works

Fixing must not break normal use. The fixed program was also checked by hand with valid and malicious input:

Input	Original program	Fixed program
`Ali,20,90;85;100`	Average 91.7, A	Average 91.7, A
`Ali,abc,`	ValueError - generic error	“Invalid input: age must be a whole number...”
Calculator: `2*(3+4)`	14	14
Calculator: `__import__('os')`	Executes the code	Rejected
Search: `' OR '1'='1`	Returns every student	Returns nothing (treated as a name)
Backup name: `x; whoami`	Runs a shell command	Rejected by the name check

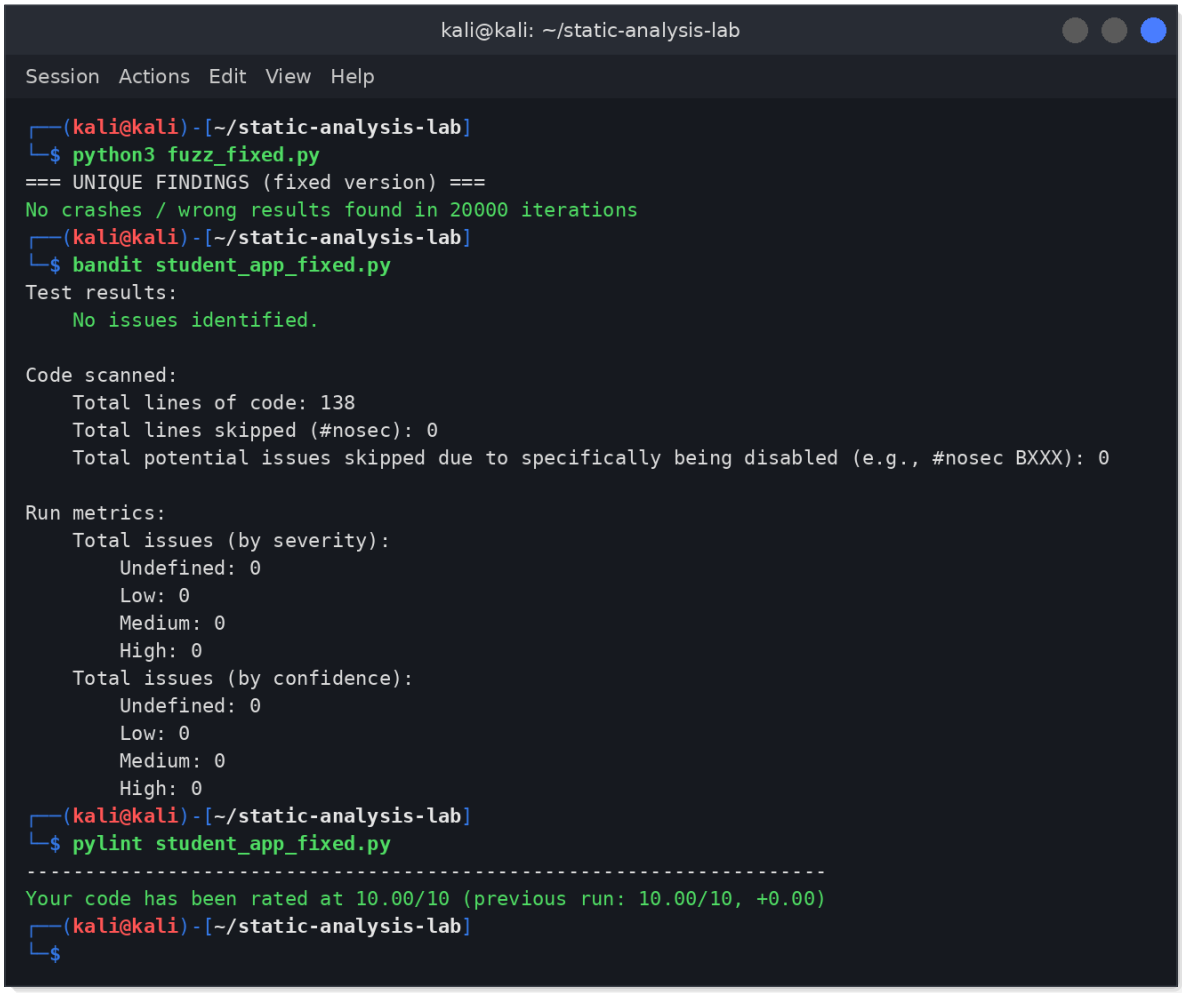


FIGURE 15 Re-test of student_app_fixed.py: no fuzzer findings, no Bandit issues and a Pylint score of 10.00 / 10.

07 Fuzzing vs Static Analysis

The most striking result of this assignment is that the two techniques found **no problem in common**. Each one was blind to everything the other one found.

Aspect	Fuzzing	Static analysis (Bandit + Pylint)
How it works	Runs the code with thousands of generated inputs	Reads the source code without running it
What it found here	8 crashes / wrong results in the core logic	9 security issues + 12 quality messages in storage and admin code
Blind spot here	Never reached SQL, eval, shell or pickle	Saw nothing wrong in parse_record, average or letter_grade
Logic (wrong-answer) bugs	Yes - when an oracle is written	No - the code is valid Python
False positives	None: each finding comes with a real input	Some: B403 / B404 only flag an import
Coverage	Only paths the inputs reach; missed score = 100	Every line and path, in seconds
Evidence	Exact failing input - easy to reproduce	File, line and CWE - no exploit input
Effort	Write a harness, choose inputs, run many iterations	One command, no set-up
Best at	Robustness and input handling	Known insecure patterns and code hygiene

Overall findings

Category	Found by fuzzing	Found by static analysis	Fixed
Crashes on bad input	7	0	7 / 7
Wrong result (silent)	1	0	1 / 1
Security weaknesses	0	9 (Bandit)	9 / 9
Code-quality messages	0	12 (Pylint)	12 / 12

VERDICT

Neither technique alone would have been enough. Fuzzing proved, with concrete inputs, that the program falls over or lies on unexpected data. Static analysis pointed straight at the injection, deserialisation and cryptography mistakes that random input almost never triggers. Used together - and followed by fixing and re-testing - they took the program from **8 crashes, 9 security issues and 8.45 / 10** to **0, 0 and 10 / 10**.

08 Conclusion & Lessons Learned

- **Validate input at the boundary.** Seven of the eight fuzzer findings came from trusting the format of user input. One validation step in parse_record removed all of them.
- **Never build code, SQL or shell commands from strings.** Parameterised queries, an AST evaluator and shutil remove whole classes of injection instead of patching single cases.
- **Crash-free does not mean correct.** The negative-score bug appeared only because the fuzzer had an oracle that knew the expected answer.
- **Read the warnings, don't just silence them.** The best fix for the pickle and subprocess warnings was to stop needing those modules, so the risky imports disappeared.
- **Re-test after fixing.** Running the same seed and the same tools again is what proves that the fixes work and that valid input still gives the same answers.

Submitted files

File	Contents
student_app.py	Part 1 - the original program with planted bugs and weaknesses
fuzz_task_1.py	Part 2 - the random fuzzer
bandit_report.txt / .json, pylint_report.txt	Part 2 - static analysis reports
student_app_fixed.py	Part 3 - the fixed program
fuzz_fixed.py	Part 3 - the same fuzzer run against the fixed program